# COMP3222 Machine Learning Technologies

## Lab Sheet 4 (Week 5)

**When:** Monday 19 October 2026, 13:00–15:00. **Based on:** Week 4 lectures, L4.1 Time Series Machine Learning and L4.2 Time Series Classification. **Repository:** https://github.com/TonyBagnall/COMP3222-2026

This lab practises working with collections of time series. You will compare whole-series distances and build features from summaries, intervals, shapelets and convolutions. Start with the files and run instructions in `code/lab4/README.md`. There are three tasks and an optional extension. All of them use GunPoint, the running example from the lectures. It has 50 training and 150 test cases, each a single channel of length 150.

### Task 1: Distances and nearest neighbours

Open `code/lab4/lab4_task1.py`. It loads GunPoint and the multivariate BasicMotions problem with `aeon`. Print the shapes and make sure you can explain each axis: cases, channels and timepoints (L4.1 slide 8). What is `X_train[3, 0]`, and why does GunPoint still have a channel axis? Use `plot_examples` to look at a few cases from each class.

Complete `euclidean_distance` and `dtw_distance`. You wrote both in Lab 1; the DTW algorithm is on L4.1 slides 22 and 23. Use squared pointwise differences and return the final accumulated cost without a square root, as `aeon` does. Decorate both functions with `@njit(cache=True)`. Then call `check_distances()`, which uses the pair of series on slide 22. Your values should match `aeon.distances`. Why is the DTW cost so much smaller than the squared Euclidean distance for this pair?

Next complete `OneNearestNeighbour`. In `fit`, store the training data and labels. In `predict`, return the label of the closest training case under the chosen distance. Record its test accuracy on GunPoint with Euclidean distance and with DTW. Check your results against `KNeighborsTimeSeriesClassifier(n_neighbors=1, distance="dtw")` from `aeon.classification.distance_based`.

Finally, test whether order matters (L4.1 slide 18). Fit `RandomForestClassifier` on `X_train[:, 0, :]` and record its test accuracy. Then draw one random permutation of the 150 timepoints with `np.random.default_rng(0).permutation(150)`. Apply it to both the training and test series and refit. Repeat with your 1-NN classifier using both distances. Which classifiers are unaffected by shuffling, apart from the random variation in a forest, and why? Which one depends on the order?

### Task 2: Summary and interval features

Open `code/lab4/lab4_task2.py`. A least-squares `slope` function is provided. Complete `summary_features(X)`. It should return the mean, standard deviation, minimum, maximum and slope of every channel, giving an array of shape `(n_cases, 5 * n_channels)` (L4.2 slide 9). Call `check_functions()`. It uses the series `a = [0, 0, 2, 2]` and `b = [0, 2, 0, 2]` from slide 8. Which features are equal for the two series, and which feature tells them apart? Check the shapes for GunPoint and BasicMotions.

Wrap the function in `SummaryTransformer`, then complete `RandomIntervalTransformer`. In `fit`, use `np.random.default_rng(self.random_state)` to choose `n_intervals` start and end positions, each at least `min_length` long. In `transform`, extract the mean, standard deviation and slope of every interval. Use the **same intervals for every case**, including test cases. This is the transform inside Time Series Forest (slides 16 and 17).

Use `make_pipeline` to put each transformer before a `RandomForestClassifier(random_state=0)`. Compare test accuracy on GunPoint for three inputs: the raw series, summary features and 20 random intervals. Why do whole-series summaries do so badly here? Now try `Catch22` from `aeon.transformations.collection.feature_based` in the same pipeline, and `TimeSeriesForestClassifier` from `aeon.classification.interval_based`. How does your interval pipeline differ from Time Series Forest?

### Task 3: Shapelet distance and convolution from scratch

This task is low level. You will implement the two ways of sliding a short pattern along a series from L4.2, then compare them on the same pattern in a plot. Open `code/lab4/lab4_task3.py`.

Complete `z_normalise`, `distance_profile` and `shapelet_distance` (L4.2 slide 24). At each position, `distance_profile` z-normalises the window of the series and returns its Euclidean distance to the z-normalised shapelet. A series of length m and a shapelet of length L give m − L + 1 distances. sDist is the minimum of the profile. Next complete `apply_kernel`, which returns the dot product of the kernel with the window at each position (no normalisation), and `max_and_ppv` (slide 30). Call `check_functions()`. The first window on slide 24 should normalise to `[-1.069, -0.267, 1.336]`. For the series `[1, 2, 4, 3, 1]` and kernel `[-1, 0, 1]`, the activation map should be `[3, 1, -3]`, with max 3 and PPV 2/3. Use the population standard deviation, which is NumPy's default.

Now compare the two on GunPoint. The starter takes a shapelet from `X_train[5, 0, 100:140]`; use its z-normalised values as a convolution kernel too. Choose one training case from each class, then make a figure with `plt.subplots(3, 1, sharex=True)`. Plot the two series in the top row, their distance profiles against the shapelet in the middle row, and their activation maps with the kernel in the bottom row. Where does each profile reach its minimum, and where does each activation map reach its maximum? Why does the activation map respond so strongly to the large rise near timepoint 40 in both classes? What does z-normalising every window do that the convolution does not?

Finally, compute sDist and the max activation for every training case. Plot one against the other in a scatter plot, coloured by class. Which feature separates the two classes? A shapelet is chosen to match one pattern, whereas a random kernel in ROCKET is not chosen at all. How does ROCKET make up for that (slides 31 to 36)? Compare test accuracy and fit time on GunPoint for `RocketClassifier(n_kernels=2000, random_state=0)` from `aeon.classification.convolution_based` and `RDSTClassifier(random_state=0)` from `aeon.classification.shapelet_based`. Compare them with your results from Tasks 1 and 2. RDST can take a minute or two to fit.

### Optional extension: Shapelet quality and a small ROCKET

**Shapelet quality.** Complete `information_gains` in the Task 3 file. Sort the distances to form the order line (L4.2 slides 25 to 27). Try a split point halfway between each consecutive pair, and return the information gain of each split. An `entropy` function is provided. For the eight distances on slide 25, the best split has a gain of 0.311; you computed Gini gain in the same way in Lab 2. Find the best gain for the Task 3 shapelet and for the max activation feature. Then search over a few training cases, start positions and lengths for other good shapelets.

**A small ROCKET.** Write a `RandomKernelTransformer(n_kernels, random_state)`. In `fit`, generate random kernels of length 7, 9 or 11. Draw the weights from a standard normal distribution and subtract their mean, and draw a bias from a uniform distribution on [-1, 1]. In `transform`, add the bias to each activation map, then take the max and PPV, giving two features per kernel. `np.correlate(series, kernel, mode="valid")` gives the same activation map as `apply_kernel`, but much faster. Put it in a pipeline with `StandardScaler` and `RidgeClassifierCV(alphas=np.logspace(-3, 3, 10))`, then compare with `RocketClassifier` on GunPoint. How does accuracy change as you increase `n_kernels`? If time permits, run your summary pipeline and `RocketClassifier` on the six-channel BasicMotions problem.